

Sandbox Report: A CPU-Scale Prototyping Ladder for Scheduled, Fixed-Point, and Itinerant CAROM

Working note — companion to “From Scheduled Execution to Itinerant Dynamics”

July 2026

Abstract

We report a CPU-only sandbox implementation and dry run of the CAROM prototyping ladder: one shared command-routed operator library executed under three semantics (externally scheduled, deep-equilibrium fixed point, and itinerant generalized Lotka–Volterra control). The purpose was engineering validation — flush out bugs, mistuned dynamics, and silent failure modes before committing GPU budget — not scientific measurement. The run succeeded on those terms: all three variants train stably; the itinerant control system produces and, critically, *preserves under gradient training* perfectly sequential heteroclinic itineraries (256/256); and three substantive findings emerged that reshape the experimental design: (i) a routing-bottleneck expressivity constraint tying the operator count K to the primitive instruction set and thereby to command-vocabulary design; (ii) a silent deep-equilibrium pathology in which the “fixed-point” model trains into a non-convergent unrolled network unless a contraction penalty is included — with a one-line fix verified to reverse the residual profile; and (iii) a package of six tuning rules without which the itinerant dynamics either seize, die, or start mid-channel. We record the exact configuration, quantitative results per rung, what the sandbox does and does not establish, and the questions deliberately left to GPU-scale runs.

1 Purpose and scope

The sandbox run had three goals, in priority order:

- (1) **De-risk the code path.** Verify that every component — task generator, shared operator core, the three execution wrappers, training harness — runs, learns, and checkpoints, so that a subsequent Colab run spends GPU time on science rather than debugging.
- (2) **De-risk the dynamics.** The itinerant variant couples a learned neural field to a generalized Lotka–Volterra (GLV) competition; whether this coupled system is *trainable by backpropagation at all* — gradients flowing through ~ 70 integration steps without instability, and the heteroclinic channel surviving parameter updates — was the largest a priori risk in the entire proposal.
- (3) **Discover unknown unknowns.** Toy-scale contact with the architecture tends to surface constraints invisible on paper. It did.

Explicitly out of scope: converged accuracies, variant comparisons at matched compute, statistical claims (all runs are single-seed), and the free-routing order-discovery question, which is a research problem rather than an engineering one. Where the sandbox touches these, results are reported as trends only.

2 Setup

2.1 Compute environment

CPU-only Linux container, PyTorch 2.13, no GPU. Two consequences shaped the methodology: (a) model width, batch size, and step counts were cut well below the intended Colab scale ($d=48$ vs. 64; hundreds-to-thousands of steps vs. 4–8k); (b) the container reaps background processes between tool invocations, so all training ran in foreground time-budgeted chunks with check-point/resume (model, optimizer, LR schedule, and both RNG states serialized). The resume machinery, built as a workaround, is retained in the deliverable because it makes the Colab notebook robust to session timeouts at zero cost.

2.2 Task: composition of sequence transforms

Workspace of $n = 6$ slots over $\mathbb{Z}_8$. A program is a sequence of up to $L_{\max} = 5$ command tokens drawn from a 10-token vocabulary: seven primitive whole-sequence transforms

inc ($x+1$), **dbl** ($2x$), **neg** ($-x$), **rev**, **shr** (rotate right), **swp** (swap slots 0, 1), **cms** (prefix sums),

all mod 8; two *synonym* tokens semantically identical to **inc** and **rev** (to exercise routing reuse); and **noop**. Programs of effective length $L \in \{2, \dots, 5\}$ are padded with trailing **noops**, so tensors are fixed-size while critical-path depth varies per sample; depth = effective length. Supervision: per-slot cross-entropy on the final state; chance accuracy = $1/8 = 0.125$. The generator self-verifies by replaying every sampled program symbolically.

This task is the *second* design; the first, and its failure, is a principal finding (Section 3.2).

2.3 Shared operator core

$K = 8$ operators, $d = 48$, single attention head, all K batched along a leading axis (one einsum per stage, no Python loop). Each operator: content-addressed attention with queries/keys/values over $[\text{LN}(w); e; p]$ (mutable content, immutable evidence, fixed address); a two-layer GELU transform; a sigmoid gate (bias-initialized near 0, so gates open at ~ 0.5); update proposal $u = g \odot \tanh(z)$. Routing is an embedding $A \in \mathbb{R}^{10 \times 8}$ softmaxed per command. Readout: LayerNorm–MLP per slot. Identical core instantiated once and shared by all three variants — the controlled quantity of the ladder.

2.4 The three execution wrappers

Scheduled. $w \leftarrow w + \sum_k \beta_k^t u_k$ with $\beta^t = \rho(c_t)$, $t = 0..L_{\max}-1$.

Fixed-point. Simultaneous field $F(w) = w + \frac{1}{L} \sum_m \sum_k \rho_k(c_m) u_k(w; s_m)$ with per-command step embeddings s_m injected into the attention inputs; damped iteration $w \leftarrow (1-\alpha)w + \alpha F(w)$, $\alpha = 0.5$, 8 sweeps (initially 10), trained by full BPTT through the sweeps (affordable at this scale; the implicit-gradient machinery is deliberately deferred). Relative residual $\|w_{t+1} - w_t\|/\|w_t\|$ logged per sweep.

Itinerant. Control state $a \in \mathbb{R}_{\geq 0}^M$, $M = 5$, Euler-integrated for $S = 70$ steps at $dt = 0.2$, coupled to the workspace:

$$\begin{aligned} a_m &\leftarrow \text{clamp}(a_m + dt a_m (\text{fit}_m(w) - \sum_j \rho_{mj} a_j) + \eta \xi, 0.05, 4.0), \\ w &\leftarrow w + dt (\sum_k \beta_k(a) u_k(w) - \lambda w), \quad \beta_k(a) = \sum_m a_m \rho_k(c_m), \end{aligned}$$

with fitness $\text{fit}_m = b + s \cdot (\text{MLP}(\text{pool}(w)) + \text{bias}(c_m)) - \omega m$ and per-mode fatigue f_m ($\tau_f = 6$, strength 1.5) subtracted from fitness. Tuned constants (Section 3.4): baseline $b = 3.0$, ordinal prior $\omega = 0.1$, activation floor 0.05, learned-fitness scale s initialized 0.5, per-command biases zero-initialized. Hand-built chain (rung 4): inhibition *felt by* the successor *from* its predecessor 0.35; by the predecessor from its successor 4.0; all other pairs 2.5; self 1.0; frozen. Free variant (rung 5): $\rho = \text{softplus}$ of a learned matrix initialized near-symmetric. Initial condition: $a_0 = 1.0$, all other modes at the floor (“enter the channel at station 0”).

2.5 Protocol

AdamW, one-cycle LR ($2 \cdot 10^{-3}$ peak), gradient clipping at 1.0, batch sizes 128 (scheduled), 96 (fixed-point), 48 (itinerant), sized to per-step wall-clock. Evaluation on fresh 256–512-sample batches (the task distribution is procedurally infinite; there is no train/test split issue at these exposure levels, though held-out-composition splits are a Colab matter). Single seed throughout.

3 Results

3.1 Rung 1: scheduled CAROM

Steady, monotone learning to $\approx 70\%$ **per-slot accuracy at $\sim 3.9\text{k}$ steps** (chance 12.5%), no instabilities, still improving at budget exhaustion. Establishes that the shared core, task, and harness are sound, and provides the reference trajectory for the other rungs’ early dynamics.

3.2 Finding 1: the routing-bottleneck constraint

The first task design was an SSA-style register program: commands of the form `slot5 = add(slot2, slot3)`, with source/destination addresses baked into the token — 210 distinct commands. Scheduled CAROM stalled near chance (0.24 at 600 steps, trajectory flattening). The cause is structural, not a bug. Command-only routing means each command’s semantics is a point in the convex hull of K command-blind operator functions: sharp routing realizes at most K distinct behaviors (the simplex vertices); soft routing realizes blends, and a blend of “write to slot 3” and “write to slot 5” is not a program step but a smear. Because the operators cannot see the command, they cannot disambiguate; the *realizable instruction set has cardinality* $\approx K$. Slot-addressed instruction formats multiply the primitive count by the number of address combinations and thus demand infeasibly large K .

Design rule extracted: *keep command vocabularies small and behavior-typed; let addressing be content-based (the workspace’s job), not command-encoded*. The replacement task (Section 2.2) follows this rule — and, notably, matches the original CAROM’s reported sweet spot (“composable functions over sequences”), suggesting its designer’s task choice was implicitly load-bearing. This constraint belongs in the theory note as an expressivity bound on command-only routing.

3.3 Rung 3: fixed-point CAROM, and Finding 2 (the silent DEQ pathology)

Trained to ~ 960 of 1400 steps. Two results.

Depth-graded accuracy. At the ~ 830 -step checkpoint, per-depth accuracy was

depth	2	3	4	5
acc	0.438	0.325	0.294	0.194

— shallow compositions learned first, the expected curriculum-like structure and the raw material for the sweeps-vs-depth analysis at convergence.

The pathology. At the same checkpoint the per-sweep relative residual *rose monotonically*, $0.116 \rightarrow 0.181$ over 8 sweeps: the model was not converging to any equilibrium. It had learned to exploit the sweep loop as a weight-tied unrolled network — a known deep-equilibrium failure under naive BPTT with no contraction pressure, but dangerous here because *nothing visible breaks*: the loss falls, accuracy climbs, and the model silently ceases to be the thing the experiment is about (equilibrium semantics, adaptive compute, residual-as-confidence all presuppose convergence).

The fix, verified. Adding the final sweep’s mean residual to the loss (weight 1.0) reversed the profile within ≈ 130 steps: residuals $0.080 \rightarrow 0.0845 \rightarrow 0.080$ (rise-then-decay, final $\leq$ initial) with accuracy unharmed (0.31, consistent with the pre-fix trajectory). Protocol rule extracted: *the residual-per-sweep plot is the only observable that identifies which model was actually trained; it is mandatory, not diagnostic garnish.*

3.4 Finding 3: making the GLV channel run (six tuning rules)

The itinerant control dynamics required an iterated tune before training was attempted; each failure below was observed, diagnosed, and fixed, with the untrained hand-built channel as the test article (success = dominant- mode itinerary $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$).

- (i) **Inhibition index convention.** With $\text{comp}_m = \sum_j \rho_{mj} a_j$, ρ_{mj} is inhibition *felt by* m from j . The chain asymmetry wired backwards produces a single mode seizing control permanently. (Observed: mode 2 dominant for the entire trajectory.)
- (ii) **Positive fitness baseline.** Without it, competition drives total activity to the floor and the control state dies. (Observed: all $a_m \rightarrow \text{floor}$ by mid-trajectory.) Winner equilibrium $a^* \approx b/\rho_{mm}$; $b = 3.0$, self-inhibition 1.0.
- (iii) **Channel entry by initialization.** At uniform initialization the chain structure itself disfavors mode 0: every successor enjoys a weak- inhibition discount from its predecessor, while mode 0 receives no discount and absorbs the full successor-kills-predecessor term from mode 1. Middle modes win. (Observed: best sweep configuration achieved only 2/16 correct itineraries from uniform starts.) Fix: $a_0 = 1.0$, others at floor.
- (iv) **Activation floor sets the clock.** Saddle passage time scales as $\ln(1/\text{floor})/\lambda$; a floor of 10^{-4} makes invasions consume the whole trajectory. Floor 0.05, $dt = 0.2$, $S = 70$ yields ≈ 12 steps per phase — five phases fit.

- (v) **Zero-initialized per-command fitness biases.** Random biases inject per-token variance that randomizes handoffs before learning begins. Zeroing them took the untrained channel from 23/32 to **64/64** perfect itineraries.
- (vi) **Fatigue as the anti-trap guarantee.** Retained at $(\tau_f, k) = (6, 1.5)$ after sweeping; asymmetric inhibition proposes the order, fatigue ensures no station becomes absorbing.

3.5 Rung 4: itinerant CAROM, hand-built chain, trained

With the channel frozen and everything else (operators, routing, fitness MLP, biases, scales) trained by full BPTT through all 70 integration steps, ~ 440 steps at batch 48 gave: accuracy above chance and depth-graded (0.286/0.221/0.217/0.202 for depths 2–5; 0.189 on all-depth-5 evaluation), loss decreasing throughout, and — the central result of the sandbox —

256/256 evaluation trajectories retained perfectly sequential itineraries after training.

Gradient flow through the coupled GLV–workspace system is stable at these settings (no clipping saturation, no divergence), and parameter updates do not corrode the heteroclinic channel. This was the largest a priori risk in the itinerant proposal, and it is retired at toy scale.

3.6 Rung 5 (abbreviated): free ρ

With ρ learnable from a near-symmetric initialization and only 124 training steps (the honest limit of the CPU budget), the system showed: accuracy 0.171; asymmetry $\|\rho - \rho^\top\|_F = 0.72$ (vs. ≈ 15 for the hand-built chain); mean phases visited 1.0 (winner-take-all, no handoffs); itinerary–order Kendall $\tau = 0.0$. Order discovery did *not* emerge in this budget — as expected, and reported as a null trend, not a negative result. Its value is confirmatory: the rung structure correctly isolates *order-execution* (rung 4, now established) from *order-discovery* (rung 5, the open research question).

4 What was established, what was suggested, what remains

4.1 Established (at toy scale, single seed)

- The full pipeline — generator (self-verifying), shared core, three wrappers, chunked training with exact resume — runs and learns.
- Scheduled CAROM learns the composition task strongly (70%, climbing).
- The fixed-point wrapper trains, exhibits depth-graded competence, and — with the contraction penalty — can be held to genuinely convergent behavior.
- The coupled itinerant system is trainable by backpropagation, and the heteroclinic channel is robust to training. Sequential program execution as an emergent property of a flow, rather than a clocked loop, works mechanically.
- The routing-bottleneck constraint: realizable instruction-set size $\approx K$; command vocabularies must be behavior-typed.

4.2 Suggested (trends; require GPU-scale confirmation)

- Depth-graded accuracy in both non-scheduled variants hints that the sweeps/phases align with compositional depth as theorized; the quantitative version (residual half-life vs. critical path; dwell statistics vs. $\ln(1/\eta)/\lambda$) awaits converged models.
- Early-training trajectories of all three variants are roughly comparable, weakly suggesting no variant pays a large optimization tax at matched (small) compute.
- The one-line contraction penalty appears sufficient (not merely necessary) at this scale; whether it holds at larger d , more sweeps, and full convergence is untested.

4.3 Open questions, deliberately deferred

- (Q1) **Convergence endpoints and gaps.** Do all three variants reach comparable asymptotic accuracy at matched compute? Does itinerant CAROM match scheduled once trained out?
- (Q2) **Order discovery.** Can free ρ learn chain asymmetry, and which of the candidate pressures (itinerary/entropy regularization requiring $\geq L$ phase transitions; fatigue annealing; program-length curriculum; weakly asymmetric ρ prior) is sufficient? This is the central scientific question of the itinerant proposal.
- (Q3) **Routing interpretability.** Do the two synonym tokens converge to the same operator mixture as their primitives? (Instrumented in the deliverable; never run to convergence.)
- (Q4) **Sweeps-vs-depth and dwell-time laws** at convergence, as quantitative signatures of the claimed semantics.
- (Q5) **Length/depth generalization** (train $L \leq 4$, test $L = 5$ and beyond), untouched in the sandbox since it requires dedicated retraining.
- (Q6) **Variance.** Everything above at ≥ 3 seeds.

5 Deliverables

A Colab notebook (GPU/CPU presets; all sandbox fixes and tuned constants baked in; checkpoint/resume; residual and stackplot diagnostics inline), the three source modules, and two reference figures from the sandbox: the $a(t)$ stackplots (trained chain vs. stalled free- ρ) and the residual-profile before/after the contraction penalty.